CS 432 – Databases | Assignment 4: Sharding

Group: Track 1 — Sports Club Management System (IIT Gandhinagar)

GitHub Repository:
https://github.com/1357koushik/Sharkey_Database_Project/tree/main/Assignment_4

Video Demonstration:
https://drive.google.com/file/d/1GoRzNkXrI9J8PyBh5Om6ubmZJTbAC2M1/view?usp=sharing

---

---

1. Project Overview

This assignment extends the **Sports Club Managment** database with **horizontal scaling via sharding**.

Member-centric tables include:

Table	Rows	Shard Key
Member	40	Member_ID
Player	20	Member_ID
Coach	20	Member_ID
Administrator	10	Member_ID
Booking	20	Member_ID
Attendance	20	Member_ID
Equipment_Loan	20	Member_ID
Complaint	20	Raised_By (Member_ID)
Player_Stat	20	Member_ID
Team_Roster	20	Member_ID (via Player)

Reference tables (Sport, Facility, Equipment, Team, Event) are **replicated** to every shard.

---

2. Shard Key Selection & Justification

Shard Key: `Member_ID`

Criterion	Assessment
High Cardinality	40 distinct IDs now grows with every new member registration. The numeric suffix (1–∞) ensures unbounded distinct values.
Query-Aligned	Every API endpoint that reads or mutates member data uses `Member_ID` in its `WHERE` clause — members, bookings, attendance, equipment loans, complaints, stats, player and coach lookups.
Stable	`Member_ID` is assigned at registration and never changes. It is the primary key of the `Member` table with `ON DELETE CASCADE`, so it has zero mutation risk post-insert.

Partitioning Strategy: Hash-Based

`
shard_id = int(member_id[1:]) % 3
`

Why hash-based over range or directory?
- **Hash-based** is O(1), deterministic, requires no extra storage, and distributes new IDs uniformly across all shards (every third new member goes to each shard in rotation).

Expected Distribution

Shard	Member_IDs	Count
shard_0	M03, M06, M09, M12, M15, M18, M21, M24, M27, M30, M33, M36, M39	13
shard_1	M01, M04, M07, M10, M13, M16, M19, M22, M25, M28, M31, M34, M37, M40	14
shard_2	M02, M05, M08, M11, M14, M17, M20, M23, M26, M29, M32, M35, M38	13

Maximum skew = 14 − 13 = **1 row**, which is < 4% — well within acceptable limits.

Shard Isolation Simulation

Each shard is a separate **SQLite database file** (`shard_0.db` , `shard_1.db` , `shard_2.db`), simulating distinct physical database nodes on the same host. The application opens connections
only to the shard(s) required for each operation, so no shard can access another's data except
through the router layer.

In [1]:
```
import sys, os
sys.path.insert(0, 'router')
from shard_config import expected_distribution, NUM_SHARDS, SHARD_PATHS

all_ids = [f'M{i:02d}' for i in range(1, 41)]
dist = expected_distribution(all_ids)

print(f"Hash function : shard_id = int(member_id[1:]) % {NUM_SHARDS}")
print(f"Shard paths   :")
for sid, path in SHARD_PATHS.items():
    print(f"  shard_{sid} → {os.path.basename(path)}")
print()
for sid, ids in dist.items():
    print(f"  Shard {sid} ({len(ids):2d} members): {ids}")
```

```
Hash function : shard_id = int(member_id[1:]) % 3
Shard paths   :
  shard_0 → shard_0.db
  shard_1 → shard_1.db
  shard_2 → shard_2.db

  Shard 0 (13 members): ['M03', 'M06', 'M09', 'M12', 'M15', 'M18',
'M21', 'M24', 'M27', 'M30', 'M33', 'M36', 'M39']
  Shard 1 (14 members): ['M01', 'M04', 'M07', 'M10', 'M13', 'M16',
'M19', 'M22', 'M25', 'M28', 'M31', 'M34', 'M37', 'M40']
  Shard 2 (13 members): ['M02', 'M05', 'M08', 'M11', 'M14', 'M17',
'M20', 'M23', 'M26', 'M29', 'M32', 'M35', 'M38']
```

SubTask 2: Shard Setup and Data Migration

Schema Initialisation

Each shard database receives:
- **Reference tables** (exact copies, replicated): Sport, Facility, Equipment, Team, Event
- **Partitioned tables** (prefixed `shard_N_`): member, player, coach, administrator, booking, attendance, equipment_loan, complaint, player_stat, team_roster

The prefix naming convention follows the assignment specification:
`shard_0_member`, `shard_1_booking`, `shard_2_complaint`, etc.

In [2]:
```
from router.init_shards import init_all_shards
import os

# Remove existing shards and reinitialise cleanly
shard_dir = 'shards'
for f in os.listdir(shard_dir):
    if f.endswith('.db'):
        os.remove(os.path.join(shard_dir, f))

init_all_shards()

# Confirm tables in each shard
import sqlite3
from shard_config import SHARD_PATHS
for sid, path in SHARD_PATHS.items():
    conn = sqlite3.connect(path)
    tables = conn.execute(
        "SELECT name FROM sqlite_master WHERE type='table' ORDER BY name"
    ).fetchall()
    conn.close()
    print(f"\nShard {sid} tables ({len(tables)}):")
    for t in tables:
        print(f"  {t[0]}")
```

```
  [OK] Shard 0 schema initialised
  [OK] Shard 1 schema initialised
  [OK] Shard 2 schema initialised
All 3 shard schemas ready.

Shard 0 tables (17):
  Equipment
  Event
  Event_Team
  Facility
  Sport
  Team
  shard_0_administrator
  shard_0_attendance
  shard_0_booking
  shard_0_coach
  shard_0_complaint
  shard_0_equipment_loan
  shard_0_member
  shard_0_player
  shard_0_player_stat
  shard_0_team_roster
  sqlite_sequence

Shard 1 tables (17):
  Equipment
  Event
  Event_Team
  Facility
```

```
  Sport
  Team
  shard_1_administrator
  shard_1_attendance
  shard_1_booking
  shard_1_coach
  shard_1_complaint
  shard_1_equipment_loan
  shard_1_member
  shard_1_player
  shard_1_player_stat
  shard_1_team_roster
  sqlite_sequence

Shard 2 tables (17):
  Equipment
  Event
  Event_Team
  Facility
  Sport
  Team
  shard_2_administrator
  shard_2_attendance
  shard_2_booking
  shard_2_coach
  shard_2_complaint
  shard_2_equipment_loan
  shard_2_member
  shard_2_player
  shard_2_player_stat
  shard_2_team_roster
  sqlite_sequence
```

Data Migration

In [3]:
```
from router.migrate_data import run_migration
run_migration()
```

```
Source DB : d:\2nd
SQL\project\Claude_Assignment_4_Sharding\Assignment_4\sports_club.db

── Replicating reference tables ──────────────────────────
  [REF] Sport: 20 rows replicated to all 3 shards
  [REF] Facility: 20 rows replicated to all 3 shards
  [REF] Equipment: 20 rows replicated to all 3 shards
  [REF] Team: 20 rows replicated to all 3 shards
  [REF] Event: 21 rows replicated to all 3 shards

── Partitioning member-centric tables ────────────────────
  [PART] Member: 40 rows  →  shard_0:13  shard_1:14  shard_2:13
  [PART] Player: 20 rows  →  shard_0:6  shard_1:7  shard_2:7
  [PART] Coach: 20 rows  →  shard_0:7  shard_1:7  shard_2:6
  [PART] Administrator: 10 rows  →  shard_0:3  shard_1:4  shard_2:3
  [PART] Booking: 20 rows  →  shard_0:6  shard_1:7  shard_2:7
  [PART] Attendance: 20 rows  →  shard_0:6  shard_1:7  shard_2:7
  [PART] Equipment_Loan: 20 rows  →  shard_0:6  shard_1:7  shard_2:7
  [PART] Complaint: 20 rows  →  shard_0:6  shard_1:7  shard_2:7
  [PART] Player_Stat: 20 rows  →  shard_0:6  shard_1:7  shard_2:7
  [PART] Team_Roster: 20 rows  →  shard_0:6  shard_1:7  shard_2:7
(unmapped: 0)

Migration complete.
```

Verification: Data Integrity

No Records Lost — Row Count Comparison

For every partitioned table: `source_count == sum(shard_0_count + shard_1_count + shard_2_count)`

In [4]:
```
import sqlite3
from shard_config import get_shard_conn, NUM_SHARDS

SOURCE_DB = 'sports_club.db'

tables = [
    ("Member",         "shard_{}_member"),
    ("Player",         "shard_{}_player"),
    ("Coach",          "shard_{}_coach"),
    ("Administrator",  "shard_{}_administrator"),
    ("Booking",        "shard_{}_booking"),
    ("Attendance",     "shard_{}_attendance"),
    ("Equipment_Loan", "shard_{}_equipment_loan"),
    ("Complaint",      "shard_{}_complaint"),
    ("Player_Stat",    "shard_{}_player_stat"),
    ("Team_Roster",    "shard_{}_team_roster"),
]

src = sqlite3.connect(SOURCE_DB)
print(f"{'Table':<20} {'Source':>8} {'S0':>6} {'S1':>6} {'S2':>6}
{'Total':>8} {'OK':>5}")
print("-" * 65)
all_ok = True
for src_tbl, shard_tpl in tables:
    src_n = src.execute(f"SELECT COUNT(*) FROM {src_tbl}").fetchone()[0]
    shard_ns = []
    for sid in range(NUM_SHARDS):
        conn = get_shard_conn(sid)
        n = conn.execute(f"SELECT COUNT(*) FROM
{shard_tpl.format(sid)}").fetchone()[0]
        shard_ns.append(n)
        conn.close()
    total = sum(shard_ns)
    ok = "True" if total == src_n else "False"
    if total != src_n: all_ok = False
    print(f"{src_tbl:<20} {src_n:>8} {shard_ns[0]:>6} {shard_ns[1]:>6}
{shard_ns[2]:>6} {total:>8} {ok:>5}")
src.close()
```

```
Table                  Source     S0     S1     S2    Total    OK
-----------------------------------------------------------------
Member                     40     13     14     13       40  True
Player                     20      6      7      7       20  True
Coach                      20      7      7      6       20  True
Administrator              10      3      4      3       10  True
Booking                    20      6      7      7       20  True
Attendance                 20      6      7      7       20  True
Equipment_Loan             20      6      7      7       20  True
Complaint                  20      6      7      7       20  True
Player_Stat                20      6      7      7       20  True
Team_Roster                20      6      7      7       20  True
```

4.2 No Duplicates Across Shards

In [5]:
```
from shard_config import get_shard_conn, NUM_SHARDS

all_member_ids = []
for sid in range(NUM_SHARDS):
    conn = get_shard_conn(sid)
    rows = conn.execute(f"SELECT Member_ID FROM
shard_{sid}_member").fetchall()
    ids = [r[0] for r in rows]
    all_member_ids.extend(ids)
    conn.close()

total  = len(all_member_ids)
unique = len(set(all_member_ids))
dupes  = [x for x in all_member_ids if all_member_ids.count(x) > 1]
print(f"Total Member rows across shards : {total}")
print(f"Unique Member_IDs               : {unique}")
print(f"Duplicate IDs                   : {dupes if dupes else 'None'}")
```

```
Total Member rows across shards : 40
Unique Member_IDs               : 40
Duplicate IDs                   : None
```

4.3 Every Record Is in Its Correct Shard

In [6]:
```
from shard_config import get_shard_id, get_shard_conn, NUM_SHARDS

misplaced = []
for sid in range(NUM_SHARDS):
    conn = get_shard_conn(sid)
    rows = conn.execute(f"SELECT Member_ID FROM
shard_{sid}_member").fetchall()
    for row in rows:
        mid = row[0]
        expected = get_shard_id(mid)
        if expected != sid:
            misplaced.append((mid, sid, expected))
    conn.close()

print(f"Checked all 40 Member rows for correct shard placement.")
print(f"Misplaced records: {len(misplaced)}")
if not misplaced:
    print("Every record is in its hash-correct shard.")
else:
    for mid, found, expected in misplaced:
        print(f"MISPLACED: {mid} found in shard {found}, expected shard
{expected}")

# Spot checks
print("\nSpot checks (Member_ID → expected shard):")
for mid, expected in [("M01",1), ("M02",2), ("M03",0), ("M40",1),
("M30",0)]:
    actual = get_shard_id(mid)
    sym = "True" if actual == expected else "False"
    print(f"  {sym} {mid}: int('{mid[1:]}') % 3 = {actual}  (expected
{expected})")
```

```
Checked all 40 Member rows for correct shard placement.
Misplaced records: 0
Every record is in its hash-correct shard.

Spot checks (Member_ID → expected shard):
  True M01: int('01') % 3 = 1  (expected 1)
  True M02: int('02') % 3 = 2  (expected 2)
  True M03: int('03') % 3 = 0  (expected 0)
  True M40: int('40') % 3 = 1  (expected 1)
  True M30: int('30') % 3 = 0  (expected 0)
```

SubTask 3: Query Routing

The `query_router.py` module implements the three required routing patterns.
All routing is O(1) — no lookup table, no inter-shard communication for lookups.

Lookup Queries — Single Shard

In [7]:
```
from router.query_router import get_member, get_member_bookings,
get_member_complaints

print("=== Lookup: get_member('M01') ===")
m = get_member('M01')
print(f"  Name      : {m['Name']}")
print(f"  Age       : {m['Age']}")
print(f"  Routed to : shard_{m['_routed_to_shard']}")
print(f"  Formula   : int('01') % 3 = {1} ")

print()
print("=== Lookup: get_member('M03') ===")
m = get_member('M03')
print(f"  Name      : {m['Name']}")
print(f"  Routed to : shard_{m['_routed_to_shard']}")
print(f"  Formula   : int('03') % 3 = {0} ")

print()
print("=== Lookup: get_member('M02') ===")
m = get_member('M02')
print(f"  Name      : {m['Name']}")
print(f"  Routed to : shard_{m['_routed_to_shard']}")
print(f"  Formula   : int('02') % 3 = {2} ")

print()
print("=== Lookup: get_member_bookings('M01') ===")
bookings = get_member_bookings('M01')
shards_used = set(b['_shard'] for b in bookings)
print(f"  Bookings  : {len(bookings)}")
print(f"  Shards used: {shards_used}  (only shard_1 — single-shard lookup
)")
print()
for b in bookings:
    print(f"  Booking_ID={b['Booking_ID']}  Facility={b['Facility_ID']}
Time_In={b['Time_In']}")
```

```
=== Lookup: get_member('M01') ===
  Name      : Rahul Sharma
  Age       : 18
  Routed to : shard_1
  Formula   : int('01') % 3 = 1

=== Lookup: get_member('M03') ===
  Name      : Aman Verma
  Routed to : shard_0
  Formula   : int('03') % 3 = 0

=== Lookup: get_member('M02') ===
```

```
  Name      : Neha Singh
  Routed to : shard_2
  Formula   : int('02') % 3 = 2

=== Lookup: get_member_bookings('M01') ===
  Bookings  : 1
  Shards used: {1}  (only shard_1 — single-shard lookup )

  Booking_ID=1  Facility=1  Time_In=2026-02-01 13:30:00
```

Insert Operations — Routed to Correct Shard

In [8]:
```
from router.query_router import insert_member, insert_booking
from shard_config import get_shard_conn

# Insert M41 -> 41 % 3 = 2 - shard_2
res = insert_member({
    'Member_ID': 'M41',
    'Name': 'Test Member FortyOne',
    'Gender': 'M',
    'Email': 'test_m41_demo@iitgn.ac.in',
    'Age': 23
})
print(f"insert_member('M41'): {res}")

# Verify presence in shard_2 only
for sid in range(3):
    conn = get_shard_conn(sid)
    found = conn.execute(f"SELECT Name FROM shard_{sid}_member WHERE
Member_ID='M41'").fetchone()
    conn.close()
    symbol = "True FOUND" if found else "False absent"
    print(f"  shard_{sid}: {symbol}" + (f" → {found[0]}" if found else ""))

# Insert booking for M41 - also shard_2
b_res = insert_booking({'Member_ID': 'M41', 'Facility_ID': 1, 'Time_In':
'2026-04-18 10:00:00'})
print(f"\ninsert_booking for M41: {b_res}")

# Cleanup
for sid in [2]:
    conn = get_shard_conn(sid)
    conn.execute(f"DELETE FROM shard_{sid}_booking WHERE Member_ID='M41'")
    conn.execute(f"DELETE FROM shard_{sid}_member WHERE Member_ID='M41'")
    conn.commit(); conn.close()
print("(Test records cleaned up)")
```

```
insert_member('M41'): {'ok': True, 'shard': 2, 'Member_ID': 'M41'}
  shard_0: False absent
  shard_1: False absent
  shard_2: True FOUND → Test Member FortyOne

insert_booking for M41: {'ok': True, 'shard': 2, 'Booking_ID': 21}
(Test records cleaned up)
```

Range Queries — Scatter to All Shards, Merge Results

In [9]:
```
from router.query_router import range_members_by_age,
range_bookings_by_date, range_attendance_by_date

print(" Range Query: Members aged 18–21 ")
print()
print("Query touches ALL 3 shards (members are distributed by hash)")
members = range_members_by_age(18, 21)
shards_touched = set(m['_shard'] for m in members)
print(f"Shards queried : {sorted(shards_touched)}")
print(f"Members found  : {len(members)}")
for m in members:
    print(f"  {m['Member_ID']}  {m['Name']:<20}  age={m['Age']}
shard_{m['_shard']}")
```

```
 Range Query: Members aged 18–21

Query touches ALL 3 shards (members are distributed by hash)
Shards queried : [0, 1, 2]
Members found  : 8
  M01  Rahul Sharma          age=18  shard_1
  M02  Neha Singh            age=19  shard_2
  M03  Aman Verma            age=20  shard_0
  M04  Priya Mehta           age=21  shard_1
  M21  Coach Arvind          age=18  shard_0
  M22  Coach Meena           age=19  shard_1
  M23  Coach Rakesh          age=20  shard_2
  M24  Coach Nitin           age=21  shard_0
```

In [10]:
```
print("Range Query: Bookings 2026-02-01 to 2026-02-10")
print()
bookings = range_bookings_by_date("2026-02-01", "2026-02-10")
shards_touched = set(b['_shard'] for b in bookings)
print(f"Shards queried : {sorted(shards_touched)}")
print(f"Bookings found : {len(bookings)}  (merged and sorted by Time_In)")
for b in bookings:
    print(f"  Booking_ID={b['Booking_ID']:>2}  Member={b['Member_ID']}  "
          f"Time_In={b['Time_In']}  shard_{b['_shard']}")
```

```
Range Query: Bookings 2026-02-01 to 2026-02-10

Shards queried : [0, 1, 2]
Bookings found : 10  (merged and sorted by Time_In)
  Booking_ID= 1  Member=M01  Time_In=2026-02-01 13:30:00  shard_1
  Booking_ID= 2  Member=M02  Time_In=2026-02-02 14:30:00  shard_2
  Booking_ID= 3  Member=M03  Time_In=2026-02-03 15:30:00  shard_0
  Booking_ID= 4  Member=M04  Time_In=2026-02-04 13:30:00  shard_1
  Booking_ID= 5  Member=M05  Time_In=2026-02-05 14:30:00  shard_2
  Booking_ID= 6  Member=M06  Time_In=2026-02-06 15:30:00  shard_0
  Booking_ID= 7  Member=M07  Time_In=2026-02-07 13:30:00  shard_1
  Booking_ID= 8  Member=M08  Time_In=2026-02-08 14:30:00  shard_2
  Booking_ID= 9  Member=M09  Time_In=2026-02-09 15:30:00  shard_0
  Booking_ID=10  Member=M10  Time_In=2026-02-10 13:30:00  shard_1
```

---

6. Full Automated Verification (35 checks)

In [3]:
```
import subprocess
import sys
import os
from pathlib import Path

def run_script(folder_path, script_name):
    folder = Path(folder_path).resolve()
    script = folder / script_name

    if not script.exists():
        print(f"ERROR: File not found -> {script}")
        return

    try:
        env = {**os.environ, "PYTHONIOENCODING": "utf-8"}
        result = subprocess.run(
            [sys.executable, str(script)],
            cwd=folder,
            capture_output=True,
            text=True,
            encoding="utf-8",
            errors="replace",
            env=env
        )

        print("=== STDOUT ===")
        print(result.stdout if result.stdout else "[empty]")

        print("=== STDERR ===")
        print(result.stderr if result.stderr else "[empty]")

        print(f"=== EXIT CODE: {result.returncode} ===")

    except Exception as e:
        print(f"Execution failed: {e}")

run_script("tests", "verify_sharding.py")
```

```
=== STDOUT ===

== A. Data Integrity — Row Counts ========================
  [92mPASS[0m  Member: source=40 == shards_total=40
         (source: 40, across 3 shards: 40)
  [92mPASS[0m  Booking: source=20 == shards_total=20
         (source: 20, across 3 shards: 20)
  [92mPASS[0m  Attendance: source=20 == shards_total=20
         (source: 20, across 3 shards: 20)
  [92mPASS[0m  Equipment_Loan: source=20 == shards_total=20
         (source: 20, across 3 shards: 20)
  [92mPASS[0m  Complaint: source=20 == shards_total=20
         (source: 20, across 3 shards: 20)
  [92mPASS[0m  Player: source=20 == shards_total=20
         (source: 20, across 3 shards: 20)
```

```
  [92mPASS[0m  Coach: source=20 == shards_total=20
        (source: 20, across 3 shards: 20)
  [92mPASS[0m  Administrator: source=10 == shards_total=10
        (source: 10, across 3 shards: 10)
  [92mPASS[0m  Team_Roster: source=20 == shards_total=20
        (source: 20, across 3 shards: 20)

== B. No Cross-Shard Duplicates ==========================
  [92mPASS[0m  Member: 40 total rows, 40 unique IDs (no duplicates)
        Duplicate IDs: []
  [92mPASS[0m  Complaint: 20 total, 20 unique (no duplicates)

== C. Shard Assignment Correctness =======================
  [92mPASS[0m  All Member rows are in their correct shard (hash
verified)
        No misplacements found
  [92mPASS[0m    Hash check: M01 → shard 1 (expected 1)
  [92mPASS[0m    Hash check: M03 → shard 0 (expected 0)
  [92mPASS[0m    Hash check: M02 → shard 2 (expected 2)
  [92mPASS[0m    Hash check: M40 → shard 1 (expected 1)
  [92mPASS[0m    Hash check: M30 → shard 0 (expected 0)

== D. Lookup Query Routing ===============================
  [92mPASS[0m  get_member('M01') returns record from shard 1
        Result: {'Member_ID': 'M01', 'Name': 'Rahul Sharma',
'Gender': 'M', 'Email': 'user1@iitgn.ac.in', 'Phone_Number':
'900000001', 'Age': 18, 'DOB': None, 'Image': None, '_shard_id': 1,
'_routed_to_shard': 1}
  [92mPASS[0m  get_member('M03') returns record from shard 0
        Result: {'Member_ID': 'M03', 'Name': 'Aman Verma', 'Gender':
'M', 'Email': 'user3@iitgn.ac.in', 'Phone_Number': '900000003',
'Age': 20, 'DOB': None, 'Image': None, '_shard_id': 0,
'_routed_to_shard': 0}
  [92mPASS[0m  get_member('M02') returns record from shard 2
        Result: {'Member_ID': 'M02', 'Name': 'Neha Singh', 'Gender':
'F', 'Email': 'user2@iitgn.ac.in', 'Phone_Number': '900000002',
'Age': 19, 'DOB': None, 'Image': None, '_shard_id': 2,
'_routed_to_shard': 2}
  [92mPASS[0m  get_member_bookings('M01') touches exactly 1 shard
        Shards touched: {1}, bookings found: 1
  [92mPASS[0m  get_member('M99') returns None for non-existent
member

== E. Insert Routing =====================================
  [92mPASS[0m  insert_member('M41') routed to shard 2 (41 % 3 = 2)
        Result: {'ok': True, 'shard': 2, 'Member_ID': 'M41'}
  [92mPASS[0m  M41 exists in shard 2 after insert
  [92mPASS[0m  M41 does NOT exist in shard 0
  [92mPASS[0m  M41 does NOT exist in shard 1
  [92mPASS[0m  insert_booking for M41 routed to shard 2
        Result: {'ok': True, 'shard': 2, 'Booking_ID': 31}
  [92mPASS[0m  insert_member('M42') routed to shard 0 (42 % 3 = 0)
        Result: {'ok': True, 'shard': 0, 'Member_ID': 'M42'}
```

```
== F. Range Query Routing (Scatter-Gather) ================
  [92mPASS[0m  range_members_by_age(18, 22) spans 3 shards
        Members found: 10  Shards touched: [0, 1, 2]
  [92mPASS[0m  range_members_by_age result is sorted by Member_ID
  [92mPASS[0m  range_bookings_by_date('2026-02-01','2026-02-20')
spans 3 shards
        Bookings found: 20  Shards touched: [0, 1, 2]
  [92mPASS[0m  range_bookings_by_date result is sorted by Time_In
  [92mPASS[0m  range_attendance_by_date spans 3 shards (all
expected)
        Records found: 20  Shards: [0, 1, 2]
  [92mPASS[0m  list_all_members() pulls from all 3 shards
        Total members across shards: 40  Shards: [0, 1, 2]
  [92mPASS[0m  range_members_by_age(18, 18) returns exactly M01 and
M21 (spans 2 shards)
        IDs: ['M01', 'M21']  Shards: [0, 1]

== SUMMARY ================================================
  35/35 checks passed   0 failed

=== STDERR ===
[empty]
=== EXIT CODE: 0 ===
```

7. SubTask 4: Scalability & Trade-offs Analysis

Horizontal vs. Vertical Scaling

Vertical scaling means upgrading a single database server—more CPU, more RAM, faster storage. It's straightforward but capped by hardware limits. Once the machine hits its maximum capacity, scaling stops. It also introduces a single point of failure.

Horizontal scaling (sharding) distributes data across multiple independent nodes. Each node stores only part of the dataset. Capacity increases by adding more nodes instead of upgrading one. In this system, the 40-member dataset is split across three SQLite shards, so each shard handles roughly one-third of the workload for member-related operations.

As the club grows, the distribution continues consistently. Every new set of members gets evenly split across shards using the routing formula int(member_id[1:]) % NUM_SHARDS. Adding shards requires updating this logic and performing re-sharding.

Consistency

Using multiple databases introduces consistency trade-offs:

Single shard operations: Fully ACID-compliant. SQLite guarantees atomicity, consistency, isolation, and durability within one shard.
Cross-shard operations: No distributed transaction handling exists. If an operation spans multiple shards, partial failure can leave the system inconsistent. There is no 2PC or equivalent mechanism.
Replicated reference tables: Tables like Sport, Facility, Equipment, Team, and Event are duplicated across all shards. Updates must be applied everywhere. Until all shards are updated, data can be temporarily inconsistent (eventual consistency). Writes require broadcasting to all shards.
Availability
Single shard failure: If one shard fails, all data tied to it becomes inaccessible. Other shards remain functional.
Lookup queries: Continue working for unaffected shards.
Range queries: Return incomplete results if a shard is down unless explicitly handled.
Mitigation approach: In production, each shard would have replication (primary + replica). Failure would require both to go down to lose availability.

Partition Tolerance

The system tolerates shard-level failures in limited scenarios:

Query routing can catch shard-specific failures and skip unavailable nodes during range queries.
Routing logic is deterministic and stateless, so recovery is simple. Once a shard is back, it can sync via WAL replay and resume normal operation.
Insert operations targeting a failed shard cannot proceed. They must either be queued for retry or fail explicitly. Silent failure is unacceptable.
Observations and Limitations
Re-sharding cost: Adding a new shard requires redistributing existing data. This is a full migration. In production, this needs controlled strategies like double writes to avoid downtime.
Cross-shard joins: Queries spanning shards require scatter-gather and in-memory merging. This is slower and more resource-intensive than single-node joins.
SQLite limitation: SQLite is used only as a simulation. It does not represent real distributed systems. In practice, shards would be separate database servers (e.g., PostgreSQL, MySQL). The routing layer remains unchanged.
Reference table sync: Reference data is replicated but not dynamically synchronized. Updates must be manually broadcast. A production system would centralize this via a dedicated service or managed replication.

---

8. Project File Structure

```
assignment4/
├── router/
│  ├── shard_config.py # Shard topology, routing formula, connections
│  ├── init_shards.py # Creates shard schemas (DDL)
│  ├── migrate_data.py # Migrates data from source DB to shards
│  └── query_router.py # Lookup / insert / range routing functions
├── shards/
│  ├── shard_0.db # Shard 0: M03, M06, M09, M12, ...
│  ├── shard_1.db # Shard 1: M01, M04, M07, M10, ...
│  └── shard_2.db # Shard 2: M02, M05, M08, M11, ...
├── tests/
│  └── verify_sharding.py # 35-check automated verification suite
├── sharded_app.py # Shard-aware Flask API (extends Assignment 2)
└── CS432_Assignment4_Report.ipynb # This notebook
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.